

MODUL 11

(TREES)



DESKRIPSI TEMA

Binary search tree pada dasarnya memiliki konsep yang sama dengan binary tree. Perbedaannya terletak pada root node binary search tree child kiri pada root node binary tree harus lebih kecil dari root node nilainya, dan juga sebaliknya jika dikanan harus lebih besar dari pada root node.

CAPAIAN PEMBELAJARAN MINGGUAN (SUB-CAPAIAN PEMBELAJARAN)

Mahasiswa dapat memahami dan menerapkan konsep trees, binary trees, binary search trees, dan AVL trees (C₃).

PENUNJANG PRAKTIKUM

1. Software CodeBlocks

LANGKAH-LANGKAH PRAKTIKUM

Tutorial 1 – Binary Search Tree

Salinlah dan pahami kodingan berikut ini dan dikumpulkan dalam format **tutorial_BST.c** dan tambahkan traversal tree ini berikut outputnya, kalian bebas ingin memakai traversal jenis apa.

```
#include<stdio.h>
#include<stdlib.h>

//Deklarasi struct untuk linked list
struct node
{
    int key;
    struct node *left, *right;
};

//function untuk membuat node baru pada BST
struct node *newNode(int item)
{
    struct node *temp = (struct node *)malloc(sizeof(struct node));
    temp->key = item;
    temp->left = temp->right = NULL;
    return temp;
}

// fungsi traversal

/* fungsi untuk membuat node dan memberi nilai pada BST */
struct node* insert(struct node* node, int key)
{
    /* Jika tree null, dibuat nodenya */
    if (node == NULL) return newNode(key);

    /* Jika tidak null, maka akan turun ke bawah */
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);

    /* return node jika sudah elesai */
    return node;
}
```

```
int main()
{
    /* BST yang ingin dibuat
    50
   /  \
  30   70
 /  \  /  \
20  40 60  80 */
    struct node *root = NULL;
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 20);
    insert(root, 40);
    insert(root, 70);
    insert(root, 60);
    insert(root, 80);

    // traversal tree, Bebas mau pakai traverse yang mana
    // HINT : cukup passing root.
    traversal();

    return 0;
}
```

Tutorial 2 – Expression Tree

Merupakan tree yang digunakan untuk merepresentasikan sebuah expresi matematika (mathematical expression), seperti $a+b-c/d*e^f$

Hal yang perlu diperhatikan sebelum menyalin kodingan berikut:

- Potongan koding ini akan terlihat berantakan (dan absurd) jika di salin semua secara tidak rapih (jadi salinlah dengan rapih).
- Perhatikan komentar opsional ("//opsional") pada akhir baris. Kalian boleh untuk tidak menyalin baris (atau fungsi) tersebut.
- Baris yang diberi //opsional berguna untuk memberikan penjelasan tentang proses yang sedang dikerjakan.

Salinlah potongan koding ini untuk membantu mengerti jalannya Expression Tree

```
1  #include <stdio.h>
2  #include <malloc.h>
3  #include <conio.h>
4  #include <stdbool.h>
5
6  /*
7   | Struct dari data yang akan digunakan sebagai gambaran dari expression tree
8   | Dalam contoh kali ini hanya ada sebuah char s
9   | Terdapat *left untuk menunjuk anak yang ada di kiri
10  | dan *right untuk menunjuk anak yang ada di kanan
11  */
12
13  typedef struct expTree {
14  |   char s;
15  |
16  |   struct expTree *left, *right;
17  } expTree;
18
19  /*
20  | Struct yang digunakan untuk membuat stack
21  | Di dalamnya terdapat tipe data dari data yang akan disimpan
22  | dan *next untuk menunjuk ke isi stack selanjutnya
23  */
24  typedef struct st {
25  |   expTree *n;
26  |
27  |   struct st *next;
28  } st;
```

```

30  /*
31  |   Membuat node baru menggunakan data character yang diterima
32  */
33  expTree* newNode(char v) {
34  |   expTree *temp;
35  |   temp = (expTree*) malloc(sizeof(expTree));
36  |   temp->left = temp->right = NULL;
37  |   temp->s = v;
38  |   return temp;
39  | }
40
41  /*
42  |   Fungsi untuk melakukan pengecekan apakah stack kosong atau tidak
43  */
44  int isEmpty(st *stack) {
45  |   if (stack == NULL) {
46  |       return true;
47  |   }
48  |   else {
49  |       return false;
50  |   }
51  | }

```

```

53  /*
54  |   Mengambil data paling atas yang ada di top
55  |   Data di return sesuai dengan tipe data dari fungsi
56  */
57  expTree* top(st *stack) {
58  |   if (!isEmpty(stack)) {
59  |       expTree *topStack;
60  |       topStack = stack->n;
61  |       return topStack;
62  |   }
63  |   else {
64  |       return NULL;
65  |   }
66  | }
67
68  /*
69  |   Memasukkan data ke dalam stack
70  */
71  void push(st **stack, expTree *tmp) {
72  |   st *node;
73  |   node = (st*) malloc(sizeof(st));
74  |   node->next = NULL;
75  |   node->n = tmp;
76
77  |   if (!isEmpty(*stack)) {
78  |       node->next = *stack;
79  |   }
80  |   *stack = node;
81  | }

```

```

83  /*
84     Menghapus top dari stack
85     Jika stack masih memiliki isi, top dipindahkan
86     Jika stack tidak memiliki isi, stack di NULL
87  */
88  void pop(st **stack) {
89      if (isEmpty(*stack)) {
90          *stack = NULL;
91      }
92      else {
93          st *hapus = *stack;
94          if ((*stack)->next == NULL) {
95              printf("Note: Dia data terakhir di stack!\n"); //opsional
96              free(&(*stack));
97              *stack = NULL;
98              return;
99          }
100         else {
101             printf("Note: Masih ada data di dalam stack!\n"); //opsional
102             *stack = (*stack)->next;
103             hapus->next = NULL;
104             hapus = NULL;
105             free(hapus);
106             return;
107         }
108     }
109 }

```

```

111 /*
112     Fungsi yang digunakan untuk melakukan inorder traversal
113 */
114 void inorder(expTree *t) {
115     if (t!=NULL) {
116         inorder(t->left);
117         printf("%c ", t->s);
118         inorder(t->right);
119     }
120 }
121
122 /*
123     Untuk mengecek apakah value yang diterima berupa
124     operator ('+', '-', '*', '/') atau operand (berupa angka atau variabel)
125 */
126 int isOperator(char c) {
127     if (c == '+' || c == '-' || c == '*' || c == '/' || c == '^') {
128         return true;
129     }
130     else {
131         return false;
132     }
133 }

```

```
135  /*
136     Untuk print stack
137     Karena adanya keterbatasan, hasil print paling bawah adalah top dari stack
138  */
139  void printStack(st *stack) { //opsional
140      st *temp = stack;
141      printf("isi stack sekarang (paling bawah = top):\n");
142      while (temp != NULL) {
143          printf("%c\n", temp->n->s);
144          temp = temp->next;
145      }
146  }
```

```
148  expTree* constructTree(char str[]) {
149      st *stack, tStack; //kita menggunakan stack
150      expTree *tmp, *tmp1, *tmp2;
151      char temp;
152      int i;
153
154      stack = NULL;
155      tmp = NULL;
156
157      for (i = 0; i < strlen(str); i++) {
158          if (!isOperator(str[i])) {
159              printf("Input %c bukan operator\n", str[i]); //opsional
160              tmp = newNode(str[i]);
161              printf("Input %c masuk ke stack\n", str[i]); //opsional
162              push(&stack, tmp);
163              printf("Top dari stack sekarang = %c\n", top(stack)->s); //opsional
164              printStack(stack);
165              printf("\n"); //opsional
166          }
```

```

167   else {
168       printf("%c adalah operator\n", str[i]); //opsional
169       tmp = newNode(str[i]);
170
171       tmp1 = top(stack);
172       printf("%c ditampung ke tmp1\n", tmp1->s); //opsional
173       pop(&stack);
174       printf("Stack di pop. Top di stack sekarang = %c\n", top(stack)->s); //opsional
175
176       tmp2 = top(stack);
177       printf("%c ditampung ke tmp2\n", tmp2->s); //opsional
178
179       pop(&stack);
180       if (!isEmpty(stack)) {
181           printf("Stack di pop. Top di stack sekarang = %c\n", top(stack)->s); //opsional
182       }
183       else {
184           printf("Stack sekarang kosong\n"); //opsional
185       }
186
187       tmp->right = tmp1;
188       tmp->left = tmp2;
189
190       printf("Data di tmp (parent) = %c\n", tmp->s); //opsional
191       printf("Data di tmp left    = %c\n", tmp->left->s); //opsional
192       printf("Data di tmp right   = %c\n", tmp->right->s); //opsional
193
194       push(&stack, tmp);
195       printf("Top di stack sekarang    = %c\n", top(stack)->s); //opsional
196       printf("Top left stack sekarang = %c\n", top(stack)->left->s); //opsional
197       printf("Top right stack sekarang = %c\n", top(stack)->right->s); //opsional
198
199       printStack(stack); //opsional
200       printf("\n"); //opsional
201   }
202 }

```

```

203
204     tmp = top(stack);
205     pop(&stack);
206
207     printf("Press any key to show result !");
208     getch();
209 }

```

```

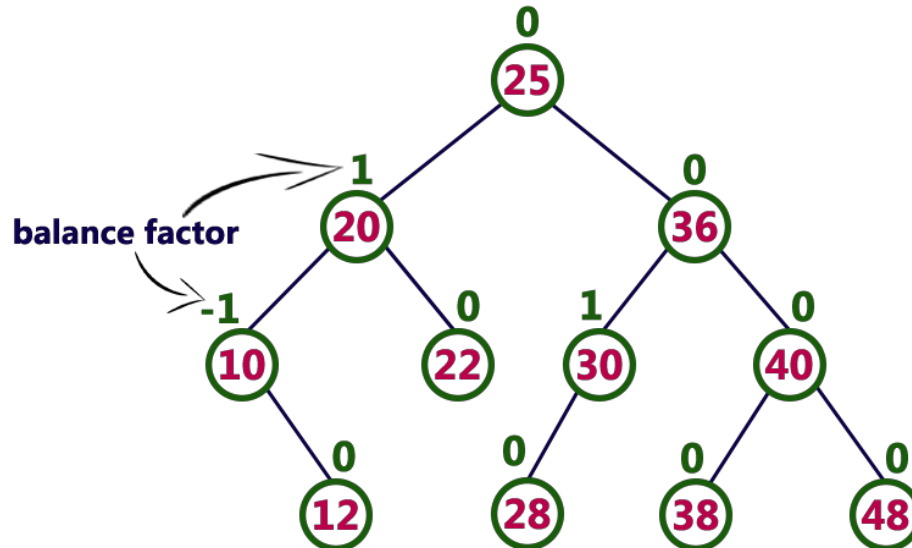
211 int main(int argc, char **argv) {
212     char postfix[] = "ab+ef*g*-";
213     printf("Waning ! Input yang diberikan harus berupa postfix dari sebuah expression\n");
214     printf("Contoh: %s\n", postfix);
215     printf("Press any key to continue. . . \n");
216     getch();
217     expTree *r = constructTree(postfix);
218     printf("infix expression is \n");
219     inorder(r);
220
221     return 0;
222 }

```

Pahamilah kodingan diatas!

Tutorial 3 – AVL Tree

AVL Tree merupakan sebuah tree yang memiliki balancing factor di setiap node-nya. Nilai dari balancing factor hanya boleh -1, 0, atau 1. Lebih dari itu, maka tree bukanlah AVL Tree.



Contoh AVL Tree dengan balancing factor tersedia

Menghitung balancing factor adalah dengan menghitung kedalaman node kiri dikurang kedalaman node kanan.

Pada contoh gambar diatas, node 25 memiliki balancing node 0 karena:

- Anak kirinya, jika di traverse (telusuri) hingga kebawah memiliki kedalaman 3.
- Anak kanannya, jika di traverse hingga kebawah, memiliki kedalaman 3.
- Sehingga jika dihitung $B = D_{kiri} - D_{kanan}$ maka didapatkan

$$B = 3 - 3 = 0$$

Dengan :

B = nilai balancing factor suatu node

D_{kiri} = nilai kedalaman terdalam node kiri

D_{kanan} = nilai kedalaman terdalam node kanan

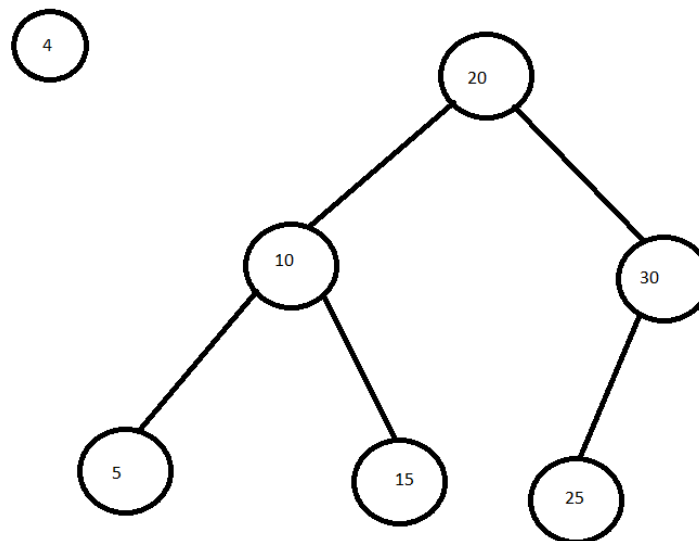
Masih pada gambar diatas, namun dengan mengambil node 20. Nilai D_{kiri} dari node 20 = 2, sedangkan nilai D_{kanan} dari node 20 = 1. Sehingga ketika dihitung $B = D_{kiri} - D_{kanan}$, didapat hasil balancing = 1 untuk node 20.

Challenge

Buatlah balancing binary tree (bukan AVL) balancing binary tree, konsepnya sama seperti binary tree.

Hanya saja balance binary tree harus balance, tidak boleh ada yang lebih dari 2 child pada parent, dan juga parent kiri tidak bisa menambah child dibawahnya sebelum child pada parent kanan sudah pada height yang sama, (NB: tidak perlu menghiraukan besarnya nilai root).

Contoh: Jika node 4 akan ditambahkan maka akan menjadi child kanan parent 30.



Program akan memiliki 5 menu, yaitu :

- Insert Data,
- View dalam pre order,
- View dalam post order,
- View dalam in order, dan
- Fitur search.

Menu tidak harus sama seperti di file pendukung, yang penting memiliki ke-5 opsi yang disebutkan di atas.

Soal Bonus:

Tambahkan menu delete node sebagai menu ke-6. (Bonus ini bersifat opsional, sehingga boleh tidak dikerjakan)

REFERENSI